

CMPUT 567: Machine Learning 2

Keven Qiu

Instructor: Martha White

Contents

1	Nonlinear Representations	3
2	Generalization Error	3
2.1	Bias and Variance for Linear Regression	3
2.2	ℓ_2 -Regularization	4
2.3	Better Generalization Error	5
3	Cross-Validation	6
4	Optimization	6
4.1	Comparisons	8
5	Generalized Linear Models	9
5.1	Poisson Example	9
5.2	General Form for GLM	9
5.3	GLM Update	10
5.4	Multinomial Logistic Regression	11
5.5	Add Regularization to GLM	12
6	Constrained Optimization With Proximal Methods	12
6.1	Proximal Gradient Descent	12
6.1.1	Box Constraints	13
6.1.2	ℓ_1 -Regularization	14
6.1.3	Linear Regression	14
7	Representation Learning	14
8	Latent Factors	14

8.1	Matrix Factorization	14
8.2	Probabilistic Approach	15
9	Neural Networks	15
9.1	Fully-Connected Feedforward Neural Networks	16
9.2	Learning f_W using Backpropagation	16
9.3	Other Network Architectures	18
10	Regularization and Auxiliary Losses on Neural Networks	18
10.1	Autoencoders	18
10.2	Supervised Autoencoder	19
11	Generative Models	20
11.1	Mixture Models	20
11.2	Variational Autoencoders	21
11.2.1	Learning VAE	21
11.2.2	Using VAE	23
12	Evaluating Generative Models	23
13	Missing Data	24
13.1	Matrix Completion	24
13.2	Direct Prediction	25
14	Uncertainty Estimation	25
14.1	Bayesian Linear Regression	26
15	Temporal Data	26
15.1	Fixed Length History	26
15.2	Recurrent Neural Networks	26

1 Nonlinear Representations

RBFs use distance to a center. The RBF is

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{c}_j\|_2^2\right)$$

It is close to 1 if $\mathbf{x}$ is similar to $\mathbf{c}_j$, 0 otherwise.

Prototype representations use any similarity measure $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ to produce $k(\mathbf{x}, \mathbf{c}_j)$.

2 Generalization Error

The generalization error for a given f , with squared error is

$$E_{x,y}[(f(X) - Y)^2] = \underbrace{E[(f(X) - f^*(X))^2]}_{\text{reducible error}} + \underbrace{E[(f^*(X) - Y)^2]}_{\text{irreducible error}}$$

where $f^*(x) = E[Y|x]$ and the irreducible error is $E_x[\text{Var}(Y|X)]$.

Bias and variance is about the function class $\mathcal{F}$ (and algorithm, i.e. objective and optimizer).

$$f_{\mathcal{D}} = \text{Alg}(\mathcal{D})$$

where $\mathcal{D} \sim \underbrace{p(x_1, y_1)p(x_2, y_2) \cdots p(x_n, y_n)}_{p(\mathcal{D})}$. Then

$$E_{\mathcal{D}}[GE(f_{\mathcal{D}})] = \underbrace{E_{\mathcal{D}}[\text{reducible error}(f_{\mathcal{D}})]}_{\text{bias}^2 + \text{variance}} + \text{irreducible error}$$

Note there are two variances: the irreducible error and the variance of our function f .

For one x :

$$\begin{aligned} E_{\mathcal{D}}[(f_{\mathcal{D}}(x) - f^*(x))^2] &= E_{\mathcal{D}}[\underbrace{(f_{\mathcal{D}}(x) - E_{\mathcal{D}}[f_{\mathcal{D}}(x)])}_a + \underbrace{(E_{\mathcal{D}}[f_{\mathcal{D}}(x)] - f^*(x))}_b]^2 \\ &= \underbrace{b^2}_{\text{bias}^2} + \underbrace{a^2}_{\text{variance}} + 2 \underbrace{E_{\mathcal{D}}[f_{\mathcal{D}}(x) - E_{\mathcal{D}}[f_{\mathcal{D}}(x)]]}_{=0} b \\ &= (E_{\mathcal{D}}[f_{\mathcal{D}}(x)] - f^*(x))^2 + \text{Var}_{\mathcal{D}}(f_{\mathcal{D}}(x)) \end{aligned}$$

If we can represent f^* (high model complexity), then $E_{\mathcal{D}}[f_{\mathcal{D}}(x)] \approx f^*(x)$ for all x .

2.1 Bias and Variance for Linear Regression

With nonlinear representation Φ , we have by SVD

$$\Phi = U\Sigma V^T \in \mathbb{R}^{n \times p}$$

where $U = [u_1, \dots, u_n]$, with left singular vectors $u_j \in \mathbb{R}^{n \times 1}$ and $V = [v_1, \dots, v_p]$, with right singular vectors $v_j \in \mathbb{R}^{p \times 1}$, and $\Sigma \in \mathbb{R}^{n \times p}$.

We consider thin SVD, where we consider only the first p rows/columns.

$$\Sigma = \begin{bmatrix} \sigma_1 & \cdots & 0 \\ 0 & \ddots & 0 \\ 0 & \cdots & \sigma_p \\ \vdots & \vdots & \vdots \\ 0 & \cdots & 0 \end{bmatrix}, \Sigma_p = \text{diag}(\sigma_1, \dots, \sigma_p), U_p = [u_1, \dots, u_p]$$

since $U\Sigma = U_p\Sigma_p$. Note that $\Sigma^T\Sigma = \Sigma_p^2$.

$$\begin{aligned} \Phi^T\Phi &= V\Sigma^T U^T U \Sigma V^T \\ &= V\Sigma^T \Sigma V^T \\ &= V\Sigma_p^2 V^T \\ \mathbf{w} &= (\Phi^T\Phi)^{-1}\Phi^T y \\ &= V\Sigma_p^{-2} V^T V\Sigma_p U_p^T y \\ &= V\Sigma_p^{-1} U_p^T y \\ &= \sum_{j=1}^p \frac{u_j^T y}{\sigma_j} v_j \end{aligned}$$

Note that for this, all singular values must be nonzero/positive. $\Phi^t = V\Sigma_p^{-1}U_p^T$ is known as the pseudoinverse of Φ .

If Φ has lower rank $r < p$, i.e. $\sigma_1, \dots, \sigma_r > 0, \sigma_{r+1}, \dots, \sigma_p = 0$, then $\Phi^t = V_r \Sigma_r^{-1} U_r^T$ where $V_r = [v_1, \dots, v_r]$.

Notice that $\mathbf{w}$ is sensitive to small singular values since we divide by them. Reasons why σ_j can be small:

- linearly correlated features (one dimension might not have anything)
- insufficient data

2.2 ℓ_2 -Regularization

For linear regression, the MLE is $p(y|x) = \mathcal{N}(\phi w, \sigma^2)$. MAP with Gaussian prior is $-\ln p(w_j) = \text{constant} + \frac{\lambda}{2\sigma^2} w_j^2$.

$$c(w) = \frac{1}{2} \|\Phi w - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2 \implies (\Phi^T\Phi + \lambda I)w = \Phi^T y \implies w = (\Phi^T\Phi + \lambda I)^{-1} \Phi^T y$$

Using SVD $\Phi^T\Phi + \lambda I = V(\Sigma_p^2 + \lambda I)V^T$,

$$w_{MAP} = V(\Sigma_p^2 + \lambda I)^{-1} U_p^T y = \sum_{j=1}^p \frac{\sigma_j}{\sigma_j^2 + \lambda} (u_j^T y) v_j$$

As long as large λ , the fraction does not blow up like it did without ℓ_2 -regularization.

2.3 Better Generalization Error

Realizable setting: $y \sim \mathcal{N}(\Phi w^*, \sigma^2)$ for some w^* and $f^* \in \mathcal{F}$ (f^* is in function class).

Let $w(\mathcal{D})$ be our estimator.

$$MSE(w(\mathcal{D})) = E_{\mathcal{D}} \left[\sum_{j=1}^p (w_j(\mathcal{D}) - w_j^*)^2 \right] = \sum_{j=1}^p (E[w_j(\mathcal{D})] - w_j^*)^2 + \sum_{j=1}^p \text{Var}(w_j(\mathcal{D}))$$

Notice $y = \Phi w^* + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. For MLE,

$$\begin{aligned} E[w_{MLE}(\mathcal{D})] &= E[(\Phi^T \Phi)^{-1} \Phi^T y] \\ &= E[(\Phi^T \Phi)^{-1} \Phi^T (\Phi w^* + \varepsilon)] \\ &= E[(\Phi^T \Phi)^{-1} (\Phi^T \Phi) w^*] + \underbrace{E[(\Phi^T \Phi)^{-1} \Phi^T] E[\varepsilon]}_{=0} \\ &= w^* \end{aligned}$$

This is unbiased as it equals w^* . For MLE variance,

$$\sum_{j=1}^p \text{Var}(w_{MLE}(\mathcal{D})) = \sigma^2 E \left[\sum_{j=1}^p \sigma_j^{-2} \right]$$

where first σ^2 is variance, not singular value. This can be very big if σ_p is often small.

For MAP,

$$E[w_{MAP}(\mathcal{D})] = E[(\Phi^T \Phi + \lambda I)^{-1} \Phi^T (\Phi w^* + \varepsilon)] \neq w^*$$

so this is biased as λ increases.

$$\sum_{j=1}^p \text{Var}(w_{MAP}(\mathcal{D})) = \sigma^2 E \left[\sum_{j=1}^p \frac{\sigma_j^2}{(\sigma_j^2 + \lambda)^2} \right]$$

This is not very big if λ is large.

Non-realizable: $y = \mathcal{N}(f^*(x), \sigma^2)$, but $f^* \notin \mathcal{F}$. w_{MLE} is also biased.

Note about n : We can stick a $\frac{1}{n}$ in front of $\Phi^T \Phi$ to act like an average over outerproducts. Note $\frac{1}{n} \Phi^T \Phi = E[\Phi^T \Phi]$.

Without normalization, $\sigma_j^2 \rightarrow \infty$ as $n \rightarrow \infty$. With normalization $\frac{1}{n} \sigma_j^2 \rightarrow$ singular values for $E[\Phi^T \Phi]$.

$\sum_{j=1}^p \sigma_j^{-2} \rightarrow 0$ as $n \rightarrow \infty$, so $\text{Var}(w_{MLE}(\mathcal{D})) \rightarrow 0$ as $n \rightarrow \infty$.

Multiple outputs: $y \in \mathbb{R}^m, \phi(x)W \approx y$.

$$Y = \begin{bmatrix} y_{11} & \cdots & y_{1m} \\ \vdots & \ddots & \vdots \\ y_{n1} & \cdots & y_{nm} \end{bmatrix} = [\mathbf{y}_1 \quad \cdots \quad \mathbf{y}_m]$$

where $\mathbf{y}_i \in \mathbb{R}^n$. So

$$\mathbf{w} = \Phi^T Y = [\Phi^T \mathbf{y}_1 \quad \cdots \quad \Phi^T \mathbf{y}_m]$$

3 Cross-Validation

Learn f on $\mathcal{D}$, use CV to estimate $\text{GE}(f)$ (this involves k partitions for train/test, train f_i , average error $\frac{1}{k} \sum e_i$).

To get k train/test splits, either k -fold (partition into k chunks, one is test) or RRS (randomly pick p points without replacement, rest is train).

k -fold: bigger k means training set is closer to n , so less bias, higher variance.

RRS: bigger k or small p is less bias, higher variance. More covariance between errors and computationally expensive.

Internal CV is for hyperparameter selection, external for general performance.

4 Optimization

Find

$$\min_{w \in \mathbb{R}^p} c(w)$$

For example, $c(w) = \frac{1}{n} \sum_{i=1}^n c_i(w)$, linear regression $c_i(w) = \frac{1}{2}(\phi(x_i)w - y_i)^2$ or logistic regression $c_i(w) = -y_i \ln f_w(x_i) - (1 - y_i) \ln(1 - f_w(x_i))$.

1. Find a stationary point w_0 , $\nabla c(w_0) = 0$.
2. Check type using second derivative test ($c'_i(w_t) < 0, > 0, = 0$).

In p -dimensions, the **Hessian** matrix $H_{c(w_t)} \in \mathbb{R}^{p \times p}$, where

$$H_{c(w)}[i, j] = \frac{\partial^2 c}{\partial w_i \partial w_j}(w)$$

$$\text{E.g. } c(w) = \frac{1}{2}(w_1 + xw_2 - y)^2, \nabla c(w) = \begin{bmatrix} (w_1 + xw_2 - y) \\ (w_1 + xw_2 - y)x \end{bmatrix} = \begin{bmatrix} \frac{\partial c}{\partial w_1}(w) \\ \frac{\partial c}{\partial w_2}(w) \end{bmatrix}.$$

$$H_{c(w)} = \begin{bmatrix} \frac{\partial}{\partial w_1} \frac{\partial}{\partial w_1} c(w) & \frac{\partial}{\partial w_1} \frac{\partial}{\partial w_2} c(w) \\ \frac{\partial}{\partial w_2} \frac{\partial}{\partial w_1} c(w) & \frac{\partial}{\partial w_2} \frac{\partial}{\partial w_2} c(w) \end{bmatrix} = \begin{bmatrix} 1 & x \\ x & x^2 \end{bmatrix}$$

For $(x = 0.1, y = 2)$ and $w_0 = (0, 0)$,

$$\nabla c(w) = \begin{bmatrix} -2 \\ -0.2 \end{bmatrix}, H_{c(w_0)} = \begin{bmatrix} 1 & 0.1 \\ 0.1 & 0.01 \end{bmatrix}$$

The eigenvalues of $H_{c(w_0)}$ are $\lambda_1 \approx 1, \lambda_2 = 0$. So we get a concave up in one direction, and flat in the other direction (a taco shape).

E.g. $c(w) = \frac{1}{2}(w_1 + x_1w_2 - y_1)^2 + \frac{1}{2}(w_1 + x_2w_2 - y_2)^2$. At $(x_1 = 0.1, y_1 = 2)$ and $(x_2 = -0.3, y_2 = -1)$,

$$H_{c(w_0)} = \begin{bmatrix} 2 & -0.2 \\ -0.2 & 0.1 \end{bmatrix}$$

for $w_0 = (0, 0)$. The eigenvalues are $\lambda_1 = 6.5, \lambda_2 = 0.08$.

Better Steps: $H_{c(w)}^{-1}$ is a matrix stepsize. The second-order method is

$$w_{t+1} = w_t - H_{c(w_t)}^{-1} \nabla c(w_t)$$

The scalar version was $w_{t+1} = w_t - \frac{1}{c''(w_t)} \cdot c'(w_t)$.

Using eigenvalue decomposition, $H_{c(w_t)}^{-1} \nabla c(w_t) = U \Lambda^{-1} \underbrace{U^T \nabla c(w_t)}_{\tilde{g}_t}$. This shows that the step is inversely proportional to curvature magnitude (λ_i 's). One step size could be bad, since in regular gradient descent uses stepsize η , so Hessian allows us not to step too much on steep surfaces and more on flat surfaces.

Computational issues: Computing $H_{c(w)}^{-1}$ is $O(p^2 n)$ for linear regression. This is because $H_{c(w)} = \frac{1}{n} \sum_{i=1}^n H_{c_i(w)}$, where $H_{c_i(w)}$ takes $O(p^2)$ and you sum n times. Computing the inverse is $O(p^3)$, but for $p < n$, $p^3 < p^2 n$, so $O(p^2 n)$ overall.

An alternative is to use a vector of stepsizes $\eta_t \in \mathbb{R}^p$. So

$$H_{c(w_t)}^{-1} \approx \text{diag}(\eta_{t,1}, \eta_{t,2}, \dots, \eta_{t,p})$$

and the update is

$$w_{t+1} = w_t - \eta_t \overset{\text{element-wise product}}{\cdot} \nabla c(w_t)$$

We do not use a fixed $\eta \in \mathbb{R}^p$. We use vector stepsizes, that change with time.

Definition 4.1: Adagrad

$$\eta_{t,j} = (1 + \bar{g}_{t,j})^{-1/2}$$

where $\bar{g}_{t,j} = \bar{g}_{t-1,j} + g_{t,j}^2$ and $\bar{g}_0 = 0$.

This has nice theory, but in practice, stepsizes shrink too much since we have a monotonic decrease in stepsize.

An intuitive modification is to take an exponential average of $g_{t,j}^2$, which leads to RMSProp.

Definition 4.2: RMSProp

$$v_{t,j} = (1 - \beta)g_{t,j}^2 + \beta v_{t-1,j}, \beta > 0$$

$$\eta_{t,j} = \frac{\eta}{\sqrt{v_{t,j} + \varepsilon}} = \eta(v_{t,j} + \varepsilon)^{-1/2}$$

Also add **momentum**: with fixed stepsize η ,

$$w_{t+1} = w_t - \eta m_{t+1}, m_{t+1} = g_t + \beta_2 m_t, \beta_2 > 0$$

An alternate form is

$$w_{t+1} = w_t - \eta g_t + \beta(w_t - w_{t-1})$$

Definition 4.3: Adam

Puts RMSProp and momentum together with extra modification.

There is a bias correct for $v_{0,j} = 0$. There is an exponential average in momentum $m_{t+1} = (1 - \beta_2)g_t + \beta_2 m_t$. The last modification is

$$\text{Adam's } \frac{1}{\sqrt{v_{t,j}} + \varepsilon} \text{ vs. RMSProp's } \frac{1}{\sqrt{v_{t,j}} + \varepsilon}$$

4.1 Comparisons

Second-order updates takes $O(p^2n)$ per iteration and gradient descent takes $O(pn)$ per iteration.

Definition 4.4: Stochastic Gradient Descent (SGD)

Randomly pick $\mathcal{B}_t \subseteq \{1, \dots, n\}$ with $|\mathcal{B}_t| = b$, the gradient $\hat{g}_t = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla c_i(w_t)$,

$$w_{t+1} = w_t - \eta_t \hat{g}_t = w_t - \underbrace{\eta_t \nabla c(w_t)}_{\text{GD}} + \underbrace{\eta_t \left(\nabla c(w_t) - \frac{1}{b} \sum_{i \in \mathcal{B}} \nabla c_i(w_t) \right)}_{\text{noise term, expected value}=0}$$

The runtime is $O(pb)$.

This is much faster than $O(pn)$ for GD. $\hat{g}_t \approx g_t = \frac{1}{n} \sum_{i=1}^n \nabla c_i(w_t) = \nabla c(w)$.

How do we pick between 2nd order GD (Newton's method), 1st order GD, SGD? In terms of computations, cost = # of iterations $\times$ compute per iteration. The convergence criterion is

$$\|\nabla c(w_t)\|_2^2 \leq \varepsilon$$

for $\varepsilon > 0$.

For now, assume $\|\nabla c(w)\| \leq L$ for any w .

- GD with fixed stepsize $\eta > 0$: After t updates/iterations:

$$\|\nabla c(w_t)\|_2^2 \leq \frac{2L}{t} \left(\underbrace{c(w_0) - c(w^*)}_{\text{how far start was from optimal}} \right)$$

So, $t \geq 2L/\varepsilon \cdot (c(w_0) - c(w^*))$ or $t = O(1/\varepsilon)$. For $\varepsilon = 0.1$, then $t = 1/\varepsilon = 10$. For $\varepsilon = 0.01$, then $t = 100$. For $t = O(1/\varepsilon)$, this is sublinear convergence. Progress slows down.

GD takes $O(np/\varepsilon)$ compute to converge.

- 2nd-Order GD: $t = O(\log(1/\varepsilon))$. For $\varepsilon = 0.1$, $t = 1$. For $\varepsilon = 0.01$, $t = 2$.

2nd-order GD takes $O(np^2 \log(1/\varepsilon))$.

When is $np^2 \log(1/\varepsilon) < np/\varepsilon \implies p^2 \log(1/\varepsilon) < p/\varepsilon$, i.e. when is 2nd-order GD better than GD? For $p = 10, \varepsilon = 0.01$, $20 < 100$. For $p = 100, \varepsilon = 0.01$, $200 > 100$. So when we have a small number of weights/ p , 2nd-order is faster.

- SGD vs. GD: SGD uses minibatches $\mathcal{B}$ of size b . If constant $\eta > 0$, almost the same convergence as GD. The convergence is

$$O\left(\frac{1}{t}\right) + \frac{L\sigma_{\mathcal{B}}^2\eta}{2}$$

$\sigma_{\mathcal{B}}^2$ is related to noisiness of gradient.

If $\eta_t = \eta/\sqrt{t}$, then $O(1/\sqrt{t})$, the oscillations eventually decay.

How can we keep $O(1/t)$ SGD convergence rate, but still reduce error term to 0? You can start with small minibatches, and increase them over time.

5 Generalized Linear Models

For classification, we had a Bernoulli distribution, $y \in \{0, 1\}$, and we learned $p(y = 1|x) = \sigma(\phi w)$. For linear regression, we had a Gaussian distribution, $y \in \mathbb{R}$, and we learned $p(y|x) = \mathcal{N}(\phi w, \sigma^2)$, where $\phi w = E[Y|x]$.

Others include Poisson distribution or multinomial distribution. $p(y|x)$ is an exponential family distribution.

5.1 Poisson Example

Imagine $y \in \{0, 1, \dots\}$ such as number of accidents in a factory, conditioned on factory info x . The pmf for Poisson is

$$p(z) = \frac{\lambda^z \exp(-\lambda)}{z!}$$

for $\lambda > 0$ is mean of Poisson distribution.

$$p(y|x) = \frac{\lambda(x)^y \exp(-\lambda(x))}{y!}$$

Our goal is to learn $\lambda(x)$.

First attempt: $\lambda(x) = \phi w$, i.e. linear regression. This does not work since $\phi w < 0$, but $\lambda > 0$.

Second attempt: $\lambda(x) = \exp(\phi w)$. Getting closer.

5.2 General Form for GLM

Definition 5.1: General Form of Generalized Linear Models

$$p(y|x) = \exp(\phi w y - a(\phi w) + b(y))$$

where transfer function $g(\phi w) = a'(\phi w)$ and $E[Y|x] = g(\phi w)$.

E.g. Gaussian $a(\phi w) = \frac{1}{2}(\phi w)^2$, $g(\phi w) = a'(\phi w) = \phi w$.

$a(\theta) = \frac{1}{2}\theta^2, a'(\theta) = \theta, b(y) = -\frac{1}{2}y^2 + \ln \frac{1}{\sqrt{2\pi}}$. The general form is

$$\begin{aligned} p(y|x) &= \exp\left(\phi w y - \frac{1}{2}(\phi w)^2 - \frac{1}{2}y^2 + \ln \frac{1}{\sqrt{2\pi}}\right) \\ &= \exp\left(-\frac{1}{2}(\phi w - y)^2\right) \exp\left(\ln \frac{1}{\sqrt{2\pi}}\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(\phi w - y)^2\right) \end{aligned}$$

E.g. Bernoulli $a'(\theta) = \sigma(\theta), g(\phi w) = \sigma(\phi w)$.

E.g. Poisson $a(\theta) = \exp(\theta), a'(\theta) = \exp(\theta) = g(\theta), b(y) = -\ln y!$. The general form is

$$\begin{aligned} p(y|x) &= \exp((\phi w)y - \exp(\phi w) - \ln y!) \\ &= \exp(\phi w y) \exp(-\exp(\phi w)) \exp(-\ln y!) \\ &= \frac{\exp(\phi w)^y \exp(-\exp(\phi w))}{y!} & (e^{ab} = (e^a)^b) \\ &= \frac{\lambda(x)^y \exp(-\lambda(x))}{y!} & (\lambda(x) = g(\phi w) = \exp(\phi w)) \end{aligned}$$

5.3 GLM Update

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \phi_i = \phi(x_i)$,

$$c_i(w) = -\ln p(y_i|x_i, w) = -\ln \exp(\phi_i w y_i - a(\phi_i w) + b(y_i)) = -(\phi_i w) y_i + a(\phi_i w) + \text{constant}$$

The gradient is

$$\nabla c_i(w) = -y_i \phi_i^T + a'(\phi_i w) \phi_i^T = (a'(\phi_i w) - y_i) \phi_i^T = \underbrace{(g(\phi_i w) - y_i)}_{\text{prediction}} \phi_i^T$$

For linear regression, $(\phi_i w - y_i) \phi_i^T$, for logistic regression, $(\sigma(\phi_i w) - y_i) \phi_i^T$, and for Poisson regression, $(\exp(\phi_i w) - y_i) \phi_i^T$.

Recap on the three GLMs:

- **Linear regression:** For $y \in \mathbb{R}$

$$g(\phi w) = \phi w \approx E[Y|x]$$

Predict with $g(\phi w)$.

- **Logistic regression:** For $y \in \{0, 1\}$

$$g(\phi w) = \sigma(\phi w) \approx E[Y|x]$$

Predict $\mathbf{1}(\sigma(\phi w) > 0.5) \approx p(y = 1|x) = E[Y|x]$.

- **Poisson regression:** For $y \in \{0, 1, 2, \dots\}$

$$g(\phi w) = \exp(\phi w) \approx E[Y|x]$$

Predict $g(\phi w)$ or mode of Poisson($\lambda = g(\phi w)$).

5.4 Multinomial Logistic Regression

This is another GLM and is multiclass classification. Given an input $x \in \mathbb{R}^{1 \times d}$ and output $y \in \mathbb{R}^{1 \times m} \in \{e_1^T, e_2^T, \dots, e_m^T\}$ where e_i^T is the row elementary vector with 1 in the i th coordinate for each class. For matrices, we have $X \in \mathbb{R}^{n \times d}$ and $Y = (y_1, \dots, y_n)^T \in \mathbb{R}^{n \times m}$.

$$p(y = [y_1, \dots, y_m]) = p(y_1 = 1)^{y_1} p(y_2 = 1)^{y_2} \dots p(y_m = 1)^{y_m} = \alpha_1^{y_1} \alpha_2^{y_2} \dots \alpha_m^{y_m}$$

where $\sum_{k=1}^m \alpha_k = 1$ and $\alpha_k \geq 0$.

E.g. $p(y = [0, 1, 0, \dots, 0]) = \alpha_1^0 \alpha_2^1 \alpha_3^0 \dots \alpha_m^0 = 0 = \alpha_2$.

In general, we want the probability given x ,

$$p(y = [y_1, \dots, y_m]) = \alpha_1(x)^{y_1} \dots \alpha_m(x)^{y_m}$$

We cannot have a transfer function $\alpha_k(x) = \sigma(\phi w_k)$ as the probabilities may not sum to 1. We can normalize so the sum is 1, this leads to the softmax function.

The transfer function g is

$$\text{softmax}(\phi W) = \left[\underbrace{\frac{\exp(\phi w_1)}{\sum_{k=1}^m \exp(\phi w_k)}}_{\alpha_1(x)} \quad \dots \quad \underbrace{\frac{\exp(\phi w_m)}{\sum_{k=1}^m \exp(\phi w_k)}}_{\alpha_m(x)} \right]$$

where $\phi \in \mathbb{R}^{1 \times p}$ and $W \in \mathbb{R}^{p \times m}$.

For each x_i ,

$$p_i = \text{softmax}(\phi_i W) \in \mathbb{R}^{1 \times m}$$

So, $p_{i1} = p(y_i = 1|x_i)$, $p_{i2} = p(y_i = 2|x_i)$, and so forth.

E.g. $m = 4$, $p_i = [0.1, 0.4, 0.3, 0.2]$, $y_i = [0, 0, 1, 0]$. Select the class

$$\hat{y}_i = \arg \max_{k \in \{1, \dots, m\}} \text{softmax}(\phi_i W) = [0, 1, 0, 0] \text{ or } 2$$

even though actual y_i is 3.

Why do we get $p_i = [0.1, 0.4, 0.3, 0.2]$, rather than $p_i = [0, 0.99, 0, 0.01]$?

- The true $p(y|x_i) = p_i$ (uncertainty, irreducible error).
- Could be insufficient data or optimization (uncertainty estimation).

GLM Update

$$c_i(W) = -\ln p(y_i|x_i), \frac{\partial c_i(W)}{\partial W_{jk}} = (p_{ik} - y_{ik})\phi_{ij}$$

For all features,

$$\frac{\partial c_i(W)}{\partial W_{:k}} = (p_{ik} - y_{ik})\phi_i^T$$

For the entire matrix,

$$\frac{\partial c_i(W)}{\partial W} = [\phi_i^T(p_{i1} - y_{i1}), \dots, \phi_i^T(p_{im} - y_{im})] = \phi_i^T(p_i - y_i) = g_t \in \mathbb{R}^{p \times m}$$

The update is

$$W_{t+1} = W_t - \eta_t \circ g_t$$

where $\circ$ is element-wise product.

An extra detail: we only need to learn $w_1, \dots, w_{m-1} \in \mathbb{R}^{p \times 1}$ since we can infer $p(y_m = 1|x) = 1 - \sum_{k=1}^{m-1} p(y_k = 1|x)$. We can pivot with $w_m = 0$ and only update $w_1, \dots, w_{m-1}$. This is w_m extra degrees of freedom.

Connection to Logistic Regression: For $m = 2$, $y_{LR} = 1 \implies y = [1, 0]$ and $y_{LR} = 0 \implies y = [0, 1]$. $w_1 = w$, $w_2 = 0$. Thus, we learn one weight vector $w_1 = w$ and pivot over w_2 .

$$p(y_{LR} = 1|x) = \frac{\exp(\phi w_1)}{\exp(\phi w_1) + \exp(\phi w_2)} = \frac{\exp(\phi w)}{\exp(\phi w) + 1} = \sigma(\phi w) = \frac{1}{1 + \exp(-\phi w)}$$

We can learn w_2 with

$$p(y_{LR} = 0|x) = p(y = [0, 1]|x) = 1 - \sigma(\phi w)$$

5.5 Add Regularization to GLM

ℓ_2 -Regularization:

$$\frac{1}{n} \sum_{i=1}^n c_i(w) + \frac{\lambda}{2} \|w\|_2^2$$

where the first summand is the MLE part ($-\ln p(y|x)$) and the second summand is the Gaussian prior, so the entire thing is MAP.

Update: $w_{t+1} = w_t - \eta_t \circ \phi_i^T(p_i - y_i) - \lambda \eta_t \circ w_t$.

ℓ_1 -Regularization:

$$\frac{1}{n} \sum_{i=1}^n c_i(w) + \lambda \|w\|_1$$

where $\|w\|_1 = \sum_{j=1}^p |w_j|$. ℓ_1 is a V-shape function, so it has a stronger preference to reach $w = 0$. ℓ_2 has a strong preference for small w .

6 Constrained Optimization With Proximal Methods

6.1 Proximal Gradient Descent

We are given the optimization problem

$$\min_{w \in \mathbb{R}^p} c(w) + r(w)$$

where c is smooth (differentiable everywhere) and r is non-smooth.

E.g. ℓ_1 -regularizer $r(w) = \lambda \|w\|_1$. This is not differentiable at $w = 0$. For gradient descent, it will keep jumping back and forth between both sides, since the gradients do not get smaller.

E.g. Constraints $w \in [-1, 1]^p$,

$$r_{\text{box}}(w) = \begin{cases} 0 & \text{if } w \in [-1, 1]^p \\ \infty & \text{otherwise} \end{cases}$$

Recall gradient descent: Local Taylor series approximation

$$c(w) \approx \hat{c}_t(w) = c(w_t) + \nabla c(w_t)^T (w - w_t) + \frac{1}{2\eta_t} \|w - w_t\|_2^2$$

Then,

$$w_{t+1} = \arg \min_{w \in \mathbb{R}^p} \hat{c}_t(w), \nabla \hat{c}_t(w) = 0 \implies w_{t+1} = w_t - \eta_t \nabla c(w_t)$$

Now, we have a non-smooth $r(w)$, so

$$w_{t+1} = \arg \min_{w \in \mathbb{R}^p} \hat{c}_t(w) + r(w)$$

Using algebraic manipulation, we can get a equivalent optimization problem:

$$w_{t+1} = \arg \min_{w \in \mathbb{R}^p} \frac{1}{2} \left\| w - \underbrace{(w_t - \eta_t \nabla c(w_t))}_{\text{usual GD outcome}} \right\|_2^2 + \eta_t r(w)$$

Algorithm: Proximal Gradient Descent

1. $\tilde{w}_{t+1} = w_t - \eta_t \nabla c(w_t)$.
2. $w_{t+1} = \text{prox}_{\eta_t r}(w) = \arg \min_{w \in \mathbb{R}^p} \frac{1}{2} \|w - \tilde{w}_{t+1}\|_2^2 + \eta_t r(w)$ (proximal operator).

6.1.1 Box Constraints

For

$$r_{\text{box}}(w_j) = \begin{cases} 0 & w_j \in [a, b] \\ \infty & \text{otherwise} \end{cases}$$

we have the proximal operator

$$\text{prox}_{\eta_t r_{\text{box}}}(v) = \begin{cases} a & \text{if } v < a \\ v & \text{if } v \in [a, b] \\ b & \text{if } v > b \end{cases}$$

Derivation: Minimization is separable.

$$\frac{1}{2} \sum_{i=1}^p (w_j - \tilde{w}_{t+1,j})^2 + \eta_t \sum_{j=1}^p r_{\text{box}}(w_j)$$

Independently,

$$\min_{w_j \in \mathbb{R}} \frac{1}{2} (w_j - \tilde{w}_{t+1,j})^2 + \eta_t r_{\text{box}}(w_j)$$

The solution w is the following:

- If $\tilde{w}_{t+1,j} \in [a, b]$, then $w_j = \tilde{w}_{t+1,j}$.
- If $\tilde{w}_{t+1,j} < a$, then $w_j = a$.
- If $\tilde{w}_{t+1,j} > b$, then $w_j = b$.

6.1.2 ℓ_1 -Regularization

For $r(w) = \lambda \|w\|_1 = \lambda \ell_1(w)$,

$$\text{prox}_{\eta_t \lambda \ell_1}(\tilde{w}_{t+1,j}) = \begin{cases} \tilde{w}_{t+1,j} - \eta_t \lambda & \text{if } \tilde{w}_{t+1,j} > \eta_t \lambda \\ 0 & \text{if } |\tilde{w}_{t+1,j}| < \eta_t \lambda \\ \tilde{w}_{t+1,j} + \eta_t \lambda & \text{if } \tilde{w}_{t+1,j} < -\eta_t \lambda \end{cases}$$

6.1.3 Linear Regression

The first step is $\tilde{w}_{t+1} = w_t - \eta_t(\phi_i w_t - y_i)\phi_i^T$. The second step is

$$w_{t+1} = \text{prox}_{\eta_t \lambda \ell_1}(\tilde{w}_{t+1})$$

element-wise.

7 Representation Learning

8 Latent Factors

Identify key features (factors) $h \in \mathbb{R}^{1 \times p}$ for an observation $x \in \mathbb{R}^{1 \times d}$ so that

$$x \approx hD$$

for dictionary $D \in \mathbb{R}^{p \times d}$. This is finding a linear weighting that approximately produces the observed inputs.

We let $D_k \in \mathbb{R}^{1 \times d}$ be a representative instance that maximally has h_k .

E.g. Let $x \in \mathbb{R}^{30}$ be patient info and we $p = 5$ number of traits for the patients. We would have a dictionary $D \in \mathbb{R}^{5 \times 30}$.

Think of h as being $\phi(x)$ from before. How do we learn h and dictionary D ?

8.1 Matrix Factorization

Principal component analysis (PCA): Suppose we want $p < d$ features, i.e. dimensionality reduction. For example, if $X \in \mathbb{R}^{n \times d}$, then we want to break it up into $X = HD$, where $H \in \mathbb{R}^{n \times p}$

and $D \in \mathbb{R}^{p \times d}$. To do this, we solve the optimization problem

$$\min_{D \in \mathbb{R}^{p \times d}} \sum_{i=1}^n \|x_i - h_i D\|_2^2$$

for all h_i , where $\hat{x}_i = h_i D$ and $\hat{X} \approx X$ for $\hat{X} = HD$.

Using the SVD of $X = U\Sigma V^T$, we have $\hat{X} = U_p \Sigma_p V_p^T$. So,

$$X - \hat{X} = \sum_{j=1}^d \sigma_j u_j v_j^T - \sum_{j=1}^p \sigma_j u_j v_j^T = \sum_{j=p+1}^d \sigma_j u_j v_j^T$$

for smallest $\sigma_{p+1} \geq \dots \geq \sigma_d$.

For the PCA solution, we have $H = U_p \Sigma_p$ (magnitudes), $D = V_p^T$ (unit length), and $\hat{X} = HD$.

For new data x_{new} ,

$$h_{new} = \arg \min_{h \in \mathbb{R}^p} \|x_{new} - h_{new} D\|_2^2 = x_{new} D^T$$

Sparse coding: For $p > d$, we want a small subset of dictionary to recreate x .

$$\min_{D \in \mathbb{R}^{p \times d}} \frac{1}{n} \sum_{i=1}^n \|x_i - h_i D\|_2^2 + \lambda \|h_i\|_1 + \lambda \sum_{j=1}^p \|D_j\|_2^2$$

ℓ_1 makes many elements of h_i zero. This is why it is called sparse coding. The last ℓ_2 makes sure that D is not high in magnitude if ℓ_1 drives h_i down to 0.

We need ℓ_1 otherwise we can have the solution $H = [X, 0]$ and $D = [I, 0]^T$ and $HD = X$, but this is a nonsense solution.

For x_{new} ,

$$h_{new} = \arg \min_{h \in \mathbb{R}^p} \|x_{new} - hD\|_2^2 + \lambda \|h\|_1$$

8.2 Probabilistic Approach

We can do probabilistic PCA.

$$p(x|hD) = \mathcal{N}(\mu = hD, \sigma^2 I)$$

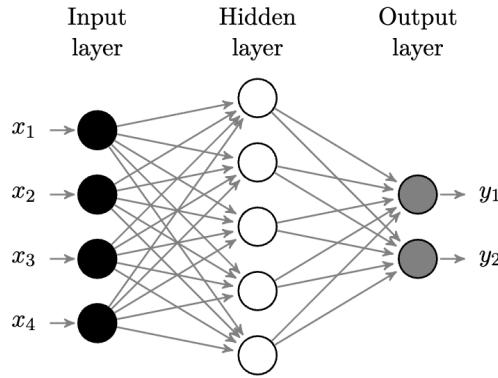
9 Neural Networks

We will use neural networks to learn representations. So suppose we have an input x and we want to learn the representation h . Each element of h can be learned as a linear weighting with x .

$$h_1 = f(xW_{:1}), h_2 = f(xW_{:2}), \dots, h_k = f(xW_{:k}) \implies h = f(xW)$$

where f is the activation function. Some examples of activation functions are

$$f(\theta) = \sigma(\theta) = \frac{1}{1 + \exp(-\theta)}, f(\theta) = \tanh(\theta) \in [-1, 1], f(\theta) = \text{ReLU}(\theta) = \max(\theta, 0)$$



What h does this operation produce? First, xW acts like a filter and tells you how similar you are to something. The activation function, such as ReLU, can then partition the space, i.e. tell you what part is active.

9.1 Fully-Connected Feedforward Neural Networks

Suppose we have a fully-connected k -layer network. We label $W^{(1)}$ from the end of network and the corresponding layer $h^{(1)}$ all the way back to the first layer $W^{(k)}$ and layer h_k .

The last layer $\hat{y} = f^{(1)}(h^{(1)}W^{(1)})$, where $f^{(1)}$ is based on properties of y (GLM choice). Each layer is like a GLM on $h^{(i)}$.

Activation function choice example: If $y \in \mathbb{R}$, then $f^{(1)} = \text{identity}$. If $y \in \{1, 2, 3\}$, $f^{(1)} = \text{softmax}$.

Each layer will have an activation function, i.e.

$$h^{(i)} = f^{(i+1)}(xW^{(i+1)})$$

The neural network is then

$$\hat{y} = f_w(x)$$

9.2 Learning f_W using Backpropagation

The backpropagation algorithm is simply gradient descent on the loss for a neural network, with a careful ordering of computation to avoid repeating computation.

1. Pick objective $y \in \mathbb{R}$, $c_i(W) = \frac{1}{2}(f_W(x_i) - y_i)^2$.

For $y \in \{0, 1\}$, $c_i(W) = y_i \ln f_W(x_i) - (1 - y_i) \ln(1 - f_W(x_i))$ and $f^{(1)} = \text{sigmoid}$.

2. Use gradient descent: need to compute $\nabla c_i(W)$.

$$W_t = (W_t^{(1)}, W_t^{(2)}, \dots, W_t^{(k)}) \implies W_{t+1} = W_t - \eta_t \nabla c_i(W_t)$$

Two Layer example: We have input x , $W^{(2)}$ to layer h_1 , $W^{(1)}$ to output.

For the output transition, $\theta^{(1)} = h^{(1)}W^{(1)}$. Let the loss be $\ell(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$.

$$\frac{\partial c_i(W)}{\partial W^{(1)}} = \frac{\partial \ell(\hat{y}_i, y_i)}{\partial W^{(1)}} = \underbrace{\frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}}}_{\delta \text{ for GLMs}} \frac{\partial \theta^{(1)}}{\partial W^{(1)}} = (f^{(1)}(\theta^{(1)}) - y_i)h_1 = \delta^{(1)}h^{(1)}$$

$$\begin{aligned} \frac{\partial c_i(W)}{\partial W_{jk}^{(2)}} &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} \frac{\partial \theta^{(1)}}{\partial W_{jk}^{(2)}} \\ &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} \frac{\sum_{i=1}^{p_1} \partial h_i^{(1)} W_i^{(1)}}{\partial W_{jk}^{(2)}} \\ &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} (0 + 0 + \dots + \frac{\partial h_k^{(1)} W_k^{(1)}}{\partial W_{jk}^{(2)}} + 0 + \dots + 0) \\ &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} \cdot W_k \frac{\partial h_k^{(1)}}{\partial W_{jk}^{(2)}} \\ &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} \cdot W_k \cdot \frac{\partial f^{(2)}(\theta_k^{(2)})}{\partial \theta_k^{(2)}} \frac{\partial \theta_k^{(2)}}{\partial W_{jk}^{(2)}} \\ &= \frac{\partial \ell(\hat{y}_i, y_i)}{\partial \theta^{(1)}} \cdot W_k \cdot f'^{(2)}(\theta_k^{(2)}) x_{ij} \end{aligned}$$

where $\theta_k^{(2)} = xW_{:k}^{(2)}$.

E.g. $f^{(2)} = \sigma$, $\sigma'(\theta) = (1 - \sigma(\theta))\sigma(\theta)$.

$f^{(2)} = \text{ReLU}$,

$$f'^{(2)} = \begin{cases} 1 & \theta > 0 \\ 0 & \text{otherwise} \end{cases}$$

Putting it all together,

$$\begin{aligned} \delta_k^{(2)} &= \delta^{(1)} W_k^{(1)} f'^{(2)}(\theta_k^{(2)}) \\ \frac{\partial c_i(W)}{\partial W_{jk}^{(2)}} &= \delta_k^{(2)} x_{ij}, \quad \frac{\partial c_i(W)}{\partial W^{(1)}} = \delta_k^{(1)} h^{(1)} \end{aligned}$$

The error for $\delta_j^{(\ell+1)}$ is

$$\delta_j^{(\ell+1)} = \left(\sum_k W_{jk}^{(\ell)} \delta_k^{(\ell)} \right) f^{(\ell+1)}(\theta_j^{(\ell+1)})$$

Some questions:

- What if there are two outputs? Before, $\delta_k^{(2)} = W_k^{(1)} \delta^{(1)} f'(\theta_k^{(2)})$, now

$$\delta_k^{(2)} = (W_{k1}^{(1)} \delta_1^{(1)} + W_{k2}^{(1)} \delta_2^{(1)}) f'(\theta_k^{(2)})$$

- What is $\delta_1^{(1)}$ and $\delta_2^{(2)}$?

For $y \in \mathbb{R}$, $f^{(1)}$ is the identity.

$$\delta_1^{(1)} = (h^{(1)}W_{:1}^{(1)} - y_{i1}), \delta_2^{(1)} = (h^{(1)}W_{:2}^{(1)} - y_{i2})$$

For $y \in \{0, 1, 2, \dots\}$, then Poisson regression which uses $f^{(1)} = \exp$, so

$$\delta_1^{(1)} = \exp(h^{(1)}W_{:1}^{(1)}) - y_{i1}, \delta_2^{(1)} = \exp(h^{(1)}W_{:2}^{(1)}) - y_{i2}$$

- Is loss per output always separately added up?

No, multinomial logistic regression.

All the above derivation is to compute the gradient $g_t = \nabla c_i(W_t)$. Now we can use g_t in any of the optimizers like SGD with fixed stepsize, Adam update, or proximal update if we have a non-smooth r .

A nuance of neural networks is that initialization of W_0 matters a lot (this is the starting point for gradient descent). In GLMs, we could initialize $w_0 = 0$, the convex function would converge to w^* . For example, if $W^{(2)} = 0$, then $h = \max(xW^{(2)}, 0) = 0$ for ReLU activation. $\delta^{(1)} = (hW^{(1)} - y)h = 0$ and $\delta_j^{(2)} = W_j^{(1)}\delta^{(1)}f'(\theta_j) = 0$. There is no update and so it is stuck.

For an ideal choice, we obviously would like to choose near optimal weights, but this is not possible. Some heuristics would be randomly initialize W where we keep variances uniform between layers.

9.3 Other Network Architectures

Definition 9.1: Sparse Neural Networks

Less weights than a fully-connected neural network.

Advantages: computational, biologically-inspired bias.

Definition 9.2: Skip Connection

Connecting nodes across layers.

This is to avoid vanishing gradients.

10 Regularization and Auxiliary Losses on Neural Networks

10.1 Autoencoders

Definition 10.1: Autoencoder

A neural network that has the same solution as PCA.

A linear autoencoder is for $x \in \mathbb{R}^d$ with $p < d$ and $W = (W^{(1)}, W^{(2)})$, we have a neural network to a hidden layer of p dimension to $\hat{x}$.

$$c_i(W) = \frac{1}{2} \left\| \underbrace{x_i W^{(2)} W^{(1)}}_{\hat{x}_i} - x_i \right\|_2^2$$

The bottleneck: $W^{(2)}W^{(1)} = \tilde{W}$ with $\text{rank}(\tilde{W}) \leq p < d$.

Using SVD,

$$\tilde{W} = U_d \Sigma_d V^T = U_d \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p, 0, \dots, 0) V^T$$

For PCA:

$$\min_D \sum_{i=1}^n \|x_i - h_i D\|_2^2$$

For Autoencoders:

$$\min_{W^{(2)}, W^{(1)}} \sum_{i=1}^n \|x_i W^{(2)} W^{(1)} - x_i\|_2^2$$

Recall the solution for PCA,

$$H = U_p \Sigma_p, D = V_p^T$$

The solution for autoencoders is

$$W^{(1)} = V_p^T, W^{(2)} = V_p$$

$$\text{Check: } XW^{(2)}W^{(1)} = U_d \Sigma_d V^T V_p V_p^T = U_d \Sigma_d \begin{bmatrix} I_p \\ 0 \end{bmatrix} V_p^T = U_p \Sigma_p V_p^T = \hat{X}.$$

Using PCA, $\min_h \|x_{\text{new}} - hD\|$. Using autoencoders, $h_{\text{new}} = x_{\text{new}} W^{(2)}$.

If you run backprop autoencoders, will you get $W^{(2)} = V_p, W^{(1)} = V_p^T$? No, $W^{(2)} Q Q^{-1} W^{(1)}$ where Q is any $p \times p$ invertible matrix and now we have a different solution.

10.2 Supervised Autoencoder

Jointly optimize representation h that maintains most information X , but also useful for predicting y . So the neural network will predict $\hat{x}$ as well as y .

$$\lambda \min_{W^{(2)}, W^{(1)}} \sum_{i=1}^n \|x_i W^{(2)} W^{(1)} - x_i\|_2^2 + \sum_{i=1}^n \ell(x_i W^{(2)} W^{(1)} y, y_i)$$

where ℓ is any GLM loss.

An alternative is to

1. Do autoencoder to get $W^{(2)}, W^{(1)}$.
2. Learn $f_{W_y^{(1)}} \approx E[Y|x]$

The loss from $\hat{x}$ is called auxiliary, since it acts like a regularization. where $W^{(1)}$ is a $p \times (d+1)$ matrix now.

11 Generative Models

11.1 Mixture Models

We learn the mixture model using the EM algorithm. We alternate between OPT. Let t be the current iteration. For the **E-step**, we have a probability array

$$p_t[i, k] = p(z_i = k | x_i, w^{(t)}) = \frac{p(x_i | z_i = k, w^{(t)}) p(z_i = k | w^{(t)})}{p(x_i | w^{(t)})}$$

using Bayes' rule. Also, $p(x_i | w^{(t)}) = \sum \alpha_k p(x_i | \theta_k^{(t)})$, so each term is

$$p(x_i | z_i = k, w^{(t)}) = p(x_i | \theta_k^{(t)}), p(z_i = k | w^{(t)}) = \alpha_k^{(t)}, p(x_i | w^{(t)}) = \sum_{k=1}^m \alpha_k^{(t)} p(x_i | \theta_k^{(t)})$$

For the **M-step**,

$$w^{(t+1)} = \arg \min_w - \sum_{i=1}^n \sum_{k=1}^m p_t[i, k] \ln p(x_i, z_i = k | w)$$

Note this does not equal $-\ln p(\mathcal{D} | w)$.

Why is this better? It breaks up into simpler optimizations.

$$\ln p(x_i, z_i = k | w) = \ln[p(x_i | z_i = k, w) p(z_i = k | w)] = \ln p(x_i | \theta_k) + \ln \alpha_k$$

The M-step objective function is then

$$\begin{aligned} & \arg \min_w - \sum_{i=1}^n \sum_{k=1}^m p_t[i, k] \ln p(x_i | \theta_k) - \sum_{i=1}^n \sum_{k=1}^m p_t[i, k] \ln \alpha_k \\ &= \arg \min_w \underbrace{\sum_{k=1}^m \sum_{i=1}^n -p_t[i, k] \ln p(x_i | \theta_k)}_{c_k(\theta_k)} - \underbrace{\sum_{i=1}^n \sum_{k=1}^m p_t[i, k] \ln \alpha_k}_{c_\alpha(\alpha)} \\ &= \arg \min_{w=(\alpha, \theta_1, \dots, \theta_m)} c_1(\theta_1) + c_2(\theta_2) + \dots + c_m(\theta_m) + c_\alpha(\alpha) \end{aligned}$$

We can minimize each $c_k(\theta_k)$ and $c_\alpha(\alpha)$ separately.

E.g. For Gaussian component, $\theta_k = (\mu_k, \Sigma_k)$. Take $\min c_k(\theta_k)$. The weighted mean is $\mu_k = \frac{1}{\sum_{i=1}^n p_t[i, k]}$.

EM algorithm: Initialize $w^{(0)}, t = 0$. m Gaussians $\mu_1^{(t)}, \dots, \mu_m^{(t)}$ set to random x_i in $\mathcal{D}$. $\Sigma_1^{(t)}, \dots, \Sigma_m^{(t)}$ diagonal with some large number, and $\alpha_1^{(t)}, \dots, \alpha_m^{(t)} = \frac{1}{m}$.

Until convergence, do

1. E-Step: Compute $p_t[i, k]$ using $w^{(t)}$.
2. M-Step: Get $w^{(t+1)}$ using $p_t[i, k]$.

11.2 Variational Autoencoders

z is a hidden discrete variable. What about continuous hidden variables? We have seen this for probabilistic PCA. For regular PCA, $p(x|w) = \int p(x|h, \mathcal{D})p(h)dh = E_{h \sim p}[p(x|h)]$. For PPCA, $p(x|h, \mathcal{D}) = \mathcal{N}(\mu = hD, \sigma^2 I)$.

For mixtures, $p(x|w) = \sum_{k=1}^m \alpha_k p(x|\theta_k) = E_{k \sim p_\alpha}[p(x|h)]$.

Instead, make $p(x|h)$ a more complex relationship between x and h .

$$p(x|h, W) = \mathcal{N}(\mu = f_W(h), \sigma^2 I)$$

where $f_W(h)$ is a neural network.

Definition 11.1: Variational Auto-Encoder (VAE)

$$p(x|W) = \int p(h)p(x|h, W)dh = E_{h \sim p}[p(x|h, W)]$$

This allows a much more complex generative model. It is also still easy to sample from:

1. Sample $h \sim \mathcal{N}(0, I)$.
2. Sample $x \sim \mathcal{N}(f_W(h), \sigma^2 I)$, $\mu = f_W(h)$ is from the forward-pass.

Often we use $\bar{x} = f_W(h)$ as our sample, rather than $x = \bar{x} + \varepsilon, \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$.

11.2.1 Learning VAE

Given $\mathcal{D} = \{x_i\}_{i=1}^n$, $p(x_i|W) = E_{h \sim p}[p(x_i|h, W)] = \int p(h)p(x_i|h, W)dh$.

$$\arg \min_W \sum_{i=1}^n -\ln p(x_i|W)$$

Approach 1: Sample $h_1, \dots, h_m \sim \mathcal{N}(0, I)$, $h_k \in \mathbb{R}^p$, and $p(x_i|W) \approx \frac{1}{m} \sum_{k=1}^m p(x_i|h_k, W)$.

Issue: need a very big m , because $p(x_i|h, W) \approx 0$ for most h .

Solution: Focus sampling h that is more pertinent to x to estimate integral.

$$p(h|x_i, W) = \frac{p(x_i|h, W)p(h)}{p(x_i|W)}$$

This follows from Bayes' rule. Approximate $q_\theta(h|x_i) \approx p(h|x_i, W)$ that is easy to sample from. We have the multivariate conditional Gaussian $q_\theta(\cdot|x_i)$, where for each h_j , $q_\theta(h_j|x_i) = \mathcal{N}(\mu_j(x), \sigma_j^2(x))$. In matrix/vector form,

$$q_\theta(h|x_i) = \mathcal{N}\left(\begin{bmatrix} \mu_1(x) \\ \vdots \\ \mu_p(x) \end{bmatrix}, \Sigma\right), \text{ where } \Sigma = \text{diag}(\sigma_1^2(x), \dots, \sigma_p^2(x))$$

Let $\theta = (\theta^{(1)}, \theta^{(2)}, \theta^{(3)})$ be the weights of the network that inputs x and a p -dimensional with mean $\mu_i(x)$ and variance $\sigma_i^2(x)$. We define the objective function that we will use to learn, using KL divergence (measures similarity between distributions)

$$\min_{\theta} \sum_{i=1}^n \text{KL}(q_{\theta}(\cdot|x_i) || p(\cdot|x_i, W))$$

where $\cdot$ means to optimize over all h . Simplifying using definition of KL divergence,

$$\begin{aligned} \int q_{\theta}(h|x) \ln \frac{q_{\theta}(h|x)}{p(h|x, W)} dh &= E_{h \sim q_{\theta}(\cdot|x)} [\ln q_{\theta}(h|x) - \ln p(h|x, W)] \\ &= E_{h \sim q_{\theta}(\cdot|x)} [\ln q_{\theta}(h|x) - \ln p(x|h, W) - \ln p(h) + \ln p(x|W)] \\ &= \underbrace{E_{h \sim q_{\theta}(\cdot|x)} [\ln q_{\theta}(h|x) - \ln \overbrace{p(h)}^{\mathcal{N}(0, I)}]}_{\text{KL}(q_{\theta}(\cdot|x) || \mathcal{N}(0, I))} - E_{h \sim q_{\theta}(\cdot|x)} [\ln p(x|h, W)] + \ln p(x|W) \end{aligned}$$

Rearranging,

$$\underbrace{\text{KL}(q_{\theta}(\cdot|x) || p(\cdot|x, W))}_{\text{encoder loss for } \theta} - \overbrace{\ln p(x|W)}^{\text{decoder loss log likelihood for } W} = \text{KL}(q_{\theta}(\cdot|x) || \mathcal{N}(0, I)) - E_{h \sim q_{\theta}(\cdot|x)} [\ln p(x|h, W)]$$

We can jointly minimize encoder and decoder loss for (W, θ) . However, the LHS is not easy to optimize. The RHS is called the $-\text{ELBO}$ (negative evidence lower bound), which is an equivalent and easier objective to optimize. So we turn to minimizing the RHS instead. Rearranging and bounding ELBO, $-\ln p(x|W) \leq -\text{elbo}(W)$.

To optimize, we use SGD.

$$\begin{aligned} -\text{elbo}_i(\theta, W) &= \text{KL}(q_{\theta}(\cdot|x_i) || \mathcal{N}(0, I)) - E_{h \sim q_{\theta}(\cdot|x_i)} [-\ln p(x_i|h_i W)] \\ -\text{elbo}(\theta, W) &= \frac{1}{n} \sum_{i=1}^n \text{elbo}_i(\theta, W) \\ &= E_{x \sim \mathcal{D}} [\text{KL}(q_{\theta}(\cdot|x) || \mathcal{N}(0, I))] - E_{x \in \mathcal{D}, h \sim q_{\theta}(\cdot|x)} [\ln p(x|h, W)] \end{aligned}$$

For decoder W , we do a stochastic sample. Sample $x_i \sim \mathcal{D}, h \sim q_{\theta}(\cdot|x_i)$, $-\nabla \ln p(x_i|h, W) = (f_W(h) - x_i) \nabla f_W(h)$. GLM $x \in \mathbb{R}^d$, $p(x|h, \mathcal{D}) = \mathcal{N}(f_W(h), \sigma^2 I)$.

For encoder θ : Part 1 (KL part). Assume $q_{\theta}(h_j|x) = \mathcal{N}(\mu_{\theta,j}(x), \sigma_{\theta,j}(x)^2)$, we have the KL objective

$$\text{KL}(q_{\theta} || \mathcal{N}(0, I)) = \sum_{j=1}^p \frac{\mu_{\theta,j}^2(x)}{2} + \frac{\sigma_{\theta,j}^2(x)}{2} - \ln \sigma_{\theta,j}^2(x) - \frac{1}{2}$$

Computing ∇_{θ} is straightforward.

Part 2 (second term): $\nabla_{\theta} E_{h \sim q_{\theta}} [\ln p(x|h, W)]$. We use the *reparameterization trick*. $h_j \in \mathcal{N}(\mu_{\theta,j}(x), \sigma_{\theta,j}^2(x)) \implies h_j = \mu_{\theta,j}(x) + \varepsilon_j \sigma_{\theta,j}(x)$ where $\varepsilon_j \in \mathcal{N}(0, 1)$.

Stochastic sample of gradient: Sample $x \in \mathcal{D}, \varepsilon \in \mathcal{N}(0, I)$, $\nabla_{\theta} -\ln p(x, h_{\theta}(x, \varepsilon), W)$.

Putting it all together, we use SGD to optimize

$$\min_{W, \theta} E_{x \in \mathcal{D}, \varepsilon \in \mathcal{N}(0, I)} [\text{KL}(q_{\theta} || \mathcal{N}(0, I)) - \ln p(x|h_{\theta}(x, \varepsilon), W)]$$

Steps:

1. Sample $x \in \mathcal{D}, \varepsilon \in \mathcal{N}(0, I)$.
2. Forward pass.
3. Backward pass.
4. Continue to encoder $\delta_j^{(4)} = \sum W_{jk}^{(3)} \delta_k^{(3)}$. For output $\mu_{\theta,j}(x)$, $\delta_1^{(5)} = \delta_1^{(4)}$

11.2.2 Using VAE

Discard the encoder $q_\theta(\cdot|x)$ and only use decoder $f_W(h)$.

1. Sample $h \sim \mathcal{N}(0, I)$.
2. Pass h in encoder to get $\hat{x}$.
3. Use $\hat{x}$ (mean) or $x \sim \mathcal{N}(f_W(h), \sigma^2 I)$.

12 Evaluating Generative Models

We have a test data set $\mathcal{D}_{test} = \{x_{n+1}, \dots, x_{n+m}\}$. The generalization error is

$$\hat{GE}(W) = \frac{1}{m} \sum_{i=1}^m -\ln p(x_i|W)$$

This is easy to compute for mixture models. Recall for mixture models, we have $p(x_i|W) = \sum_{k=1}^m \alpha_k p(x_i|\mu_k, \sigma_k^2)$.

This is harder for VAEs. $p(x_i|W) = \int p(h)p(x_i|h, W)dh = E_{h \sim p}[p(x_i|h, W)]$ where $p(h) \in \mathcal{N}(0, I)$ is easy and $p(x_i|h, W)$ (decoder) is easy, but computing the integral is hard.

The naive strategy is sample $h_1, \dots, h_b \sim \mathcal{N}(0, I)$, and estimate the integral using $\frac{1}{b} \sum_{k=1}^b p(x_i|h_k, W) \approx p(x_i|W)$. You need large b , and for every $x_i \in \mathcal{D}_{test}$, query the decoder b times. You need a large b because $p(x_i|h, W) \approx 0$ for most h_k .

A better solution is to use our encoder $q_\theta(h|x)$ to focus sampling.

$$p(x_i|W) = \int \frac{q_\theta(h|x_i)}{q_\theta(h|x_i)} p(h)p(x_i|h, W)dh = \int q_\theta(h|x_i) \frac{p(h)}{q_\theta(h|x_i)} p(h)p(x_i|h, W)dh$$

where the fraction is just 1, so it did nothing. We now sample $h \sim q_\theta(\cdot|x_i)$ and the fraction $\frac{p(h)}{q_\theta(h|x_i)}$ is the importance sampling ratio to correct sampling.

The new procedure is sample $h_1, \dots, h_b \sim q_\theta(\cdot|x_i)$. But we need to use importance sampling to correct the distribution, so we sample many $h_{k,j} \sim \mathcal{N}(\mu_j(x_i), \sigma_j^2(x_i))$ for $k = 1, \dots, b$. Then

$$\hat{GE}_i(W) = -\ln \frac{1}{b} \sum_{k=1}^b \rho_k p(x_i|h_k, W)$$

where $\rho_k = \frac{p(h_k)}{q_\theta(h_k|x_i)}$. And the total GE is

$$\hat{GE}(W) = \frac{1}{m} \sum_{x_i \in \mathcal{D}_{test}} \hat{GE}_i(W)$$

13 Missing Data

The goal is to learn the predictor f on $\mathcal{D} = \{(x_i, A_i, y_i)\}_{i=1}^n$ where A_i are the indices available and $M_i = \{1, \dots, d\} - A_i$ are the missing indices.

The typical approach is to impute missing value (complete or fill in missing values).

13.1 Matrix Completion

One way to impute missing values is to use PCA loss. Let $X \in \mathbb{R}^{n \times d}$ be our data matrix with X_{ij} missing. With PCA, we can let $X = hD$. Then, $X_{ij} = h_i D_j$, then we can solve

$$\min \sum_{i=1}^n \sum_{j=1}^d (X_{ij} - h_i D_j)^2$$

If we can get h_i and D , then we can fill in X_{ij} if it is missing. We have 3 cases

1. Learn $D, h_1, \dots, h_n$ on complete training data using the optimization problem above. Imagine you lose X_{ij} . To recover, set $X_{ij} = \hat{X}_{ij} = h_i D_j$.
2. Learn D on complete training data, get x_{new} that is missing index k . We omit the k th term of

$$h_{new} = \arg \min_h \sum_{j=1, j \neq k}^d (x_{new,j} - h D_{:j})^2$$

Now $\hat{x}_{new,k} = h_{new} D_{:k}$. In general,

$$\sum_{j \in A_i} (x_{new,j} - h D_{:j})^2$$

3. Training data not complete, how do we learn D ?

$$\min_{h_1, \dots, h_n, D} \sum_{i=1}^n \sum_{j \in A_i} (X_{ij} - h_i D_{:j})^2$$

D will be shared across n samples and h_i shared across j for each i .

For prediction, there is a two stage approach.

- (1) Complete data using above PCA approach.

$$\hat{X}[i, j] = \begin{cases} \tilde{X}[i, j] & \text{if } j \in A_i \\ \hat{X}[i, j] & \text{otherwise} \end{cases}$$

- (2) $\tilde{X}$ is our new full data matrix.

More generally, completion solved the objective

$$\min_{x_1, m_1, \dots, x_n, m_n} \min_{h_1, \dots, h_n, D} \frac{1}{n} \sum_{i=1}^n \|x_i - h_i D\|_2^2$$

where x_{i,M_i} are the missing values in x_i . Now the x_i in the objective is $\begin{bmatrix} x_{i,A_i} \\ x_{i,M_i} \end{bmatrix}$ is composed of available x_i and outer optimization x_{i,M_i} . For PCA above, outer optimization $x_{i,M_i} = h_i D_{:M_i}$.

For autoencoders (NNs), we do

$$\min_{x_{1,M_1}, \dots, x_{n,M_n}} \min_W \frac{1}{n} \sum_{i=1}^n \|f_W(x_i) - x_i\|_2^2$$

Initialize $x_{1,M_1}, \dots, x_{n,M_n} = 0$, W as usual (Xavier init). Given $x_{1,M_1}, \dots, x_{n,M_n}$, do inner optimization, i.e. update W with backpropagation on complete data. Then do outer optimization by setting $x_{i,M_i} = f_W(x_i)_{M_i}$.

Using this in practice:

1. $\tilde{X}$ with missing values, learn our autoencoder f_W on $\tilde{X}$ using the optimization objective above. Let X be the completed $\tilde{X}$.
2. (X, y) passed into any learning algorithm f_β .
3. Get x_{new} with missing indices M_i . To get $\hat{y}_{new}$,

(a) Complete x_{new} :

$$\min_{x_{new,M_i}} \|f_W(x_{new}) - x_{new}\|_2^2$$

Start with initial guess $x_{new,M_i} = 0$, then repeatedly $x_{new,M_i} = f_W(x_{new})_{M_i}$ until it stops changing. Now, $\hat{y}_{new} = f_\beta(x_{new})$ for completed x_{new} .

13.2 Direct Prediction

Learn $p(y|x_{A_i}, M_i)$.

1. Fill $x_{ij} = 0$ if $j \in M_i$ (zero impute).
2. Create missingness indicator vector m_i , where you have 1 if it is missing and 0 if not missing.
3. Pass both x and m to the network.

Let $\langle (x, m), (w_x, w_m) \rangle$, then

$$h_1 = \sigma \left(\sum_{j=1}^d x_{ij} w_{x,j} + \sum_{j=1}^d m_{ij} w_{m,j} \right) = \sum_{j \in A_i} x_{ij} w_{x,j} + \sum_{j \in M_i} w_{m,j}$$

14 Uncertainty Estimation

So far, we have learned point estimates $f_w(x) \rightarrow \hat{y}$. We want a credible interval around $f_w(x)$ to reflect uncertainty. Our goal is $p(f_w(x)|\mathcal{D})$. Let w_n be the weights produced from $\mathcal{D}$.

The credible interval would be $[\hat{y} - \varepsilon, \hat{y} + \varepsilon]$. This is epistemic uncertainty.

Our approach is get $p(w|\mathcal{D})$, then extract $p(f_w(x)|\mathcal{D})$ for any x .

14.1 Bayesian Linear Regression

Assume $p(y|x, w) = \mathcal{N}(\phi(x)w, \sigma_y^2)$. First, we will assume σ_y^2 is known.

The prior $p(w) = \mathcal{N}(0, (\sigma_y^2/\lambda)I)$ for $\lambda > 0$. By Bayes' rule,

$$p(w|\mathcal{D}_n) = \frac{p(\mathcal{D}_n|w)p(w)}{p(\mathcal{D}_n)} = \mathcal{N}(\mu_n, \Sigma_n)$$

Gaussian is a conjugate prior to Gaussian likelihood.

$\mu_n = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T y$ and $\Sigma_n = \sigma_y^2 (\Phi^T \Phi + \lambda I)^{-1}$.

For a given x , $p(f_w(x)|\mathcal{D}) = \mathcal{N}(\phi(x)\mu_n, \phi(x)\Sigma_n\phi(x)^T)$.

15 Temporal Data

We have a sequence of data over time. The history gives a lot of contextual information that can help us overcome partial observability. It is generally infeasible to use the whole history.

Two strategies are using a fixed length history of size p , giving higher precision on a shorter chunk of history, or use a recurrent neural network that summarizes infinitely far back in time, but with lower precision.

15.1 Fixed Length History

Learn $f_w([x_{t-p+1}, x_{t-p+2}, \dots, x_t]) \approx y_t$. The input to f is a pd -length vector.

1. Create dataset $\tilde{D} = \{(\tilde{x}_1 = [x_1, \dots, x_p], \tilde{y}_1 = y_p), \dots, (\tilde{x}_{n-p+1} = [x_{n-p+1}, \dots, x_n], \tilde{y}_{n-p+1} = y_n)\}$.
2. Hand $\tilde{D}$ to regression/classification algorithm.

Issues: fixed length p , smaller history p , input dimension increases to pd .

15.2 Recurrent Neural Networks

Learn a state-update function f_w where $s_t = f_w(s_{t-1}, x_t)$.

Unrolling: $s_t = f_w(f_w(s_{t-2}, x_{t-1}), x_t) = \dots$.

What does f_w look like? An example: $s_t = \text{ReLU}(s_{t-1}W_s + x_tW_x)$, where the total weights $W = (W_s, W_x)$.

The learning is driven by prediction loss, e.g. $\|\hat{y}_t - y_t\|_2^2$ and backprop.

The general form for $s_0, \dots, s_t$:

$$\Delta W = \sum_{k=2}^{t+1} \delta^{(k)} h^{(k)}$$

unrolled to the beginning of time. This is called backpropagation through time (BPTT). We are learning $\theta = (W, \beta)$. We also initialize s_0 .

$$\min_{\theta, s_0} \frac{1}{n} \sum_{t=1}^n \|\hat{y}_t(s_0, x_1, \dots, x_t, \theta) - y_t\|_2^2$$

where $\hat{y}_t(s_0, x_1, \dots, x_t, \theta) = g_\beta(f_2(f_2(\dots(s_0, x_1), x_2), \dots, x_t))$.